

Kode ini adalah sebuah **Helper Class** dalam C# untuk melakukan pemrosesan citra (*image processing*), khususnya untuk memanipulasi **Kontras Gambar**.

Kode ini menggunakan teknik pemrosesan tingkat rendah (*low-level*) dengan memanipulasi byte array secara langsung (menggunakan LockBits dan Marshal) dan mempercepat prosesnya dengan **Parallel Processing** (menggunakan semua core CPU).

Berikut adalah penjelasan mendalam per bagian dan per baris kodenya.

Bagian 1: Import & Namespace

C#

```
using System;
using System.Drawing; // Library dasar GDI+ untuk memanipulasi gambar (Bitmap)
using System.Drawing.Imaging; // Berisi enum PixelFormat, BitmapData, ImageLockMode
using System.Runtime.InteropServices; // Untuk Marshal.Copy (interaksi memori managed vs
unmanaged)
using System.Threading.Tasks; // Untuk Parallel.For (Multithreading)

namespace MiniPhotoshop.Logic.Helpers
{
    public class ContrastHelper
    {
```

- **Penjelasan:** Bagian ini memuat pustaka yang dibutuhkan. System.Drawing adalah inti pengolahan gambar di .NET Framework/Windows. System.Threading.Tasks sangat penting di sini agar pengolahan gambar berjalan cepat dengan membagi tugas ke banyak *processor core*.

Bagian 2: Method ApplyPointContrast (Aras Titik)

Method ini mengubah kontras secara **Global**. Jika satu piksel diubah, ia tidak peduli dengan piksel di sebelahnya. Ia hanya peduli pada nilai piksel itu sendiri dan faktor kontras.

C#

```

// Menerima gambar sumber dan faktor kontras (misal: 1.5 untuk menaikkan kontras)
public Bitmap ApplyPointContrast(Bitmap source, double contrastFactor)
{
    if (source == null) return null; // Cek defensif agar tidak crash jika gambar kosong

    int w = source.Width;
    int h = source.Height;

    // 1. Membuat kanvas kosong seukuran gambar asli untuk menampung hasil
    Bitmap dst = new Bitmap(w, h, PixelFormat.Format32bppArgb);

    // 2. LockBits: Mengunci memori gambar agar tidak dipindah oleh OS/GC saat kita baca.
    // Ini jauh lebih cepat daripada method GetPixel() / SetPixel().
    BitmapData srcData = source.LockBits(new Rectangle(0, 0, w, h),
        ImageLockMode.ReadOnly, PixelFormat.Format32bppArgb);

    BitmapData dstData = dst.LockBits(new Rectangle(0, 0, w, h),
        ImageLockMode.WriteOnly, PixelFormat.Format32bppArgb);

    // 3. Menghitung total byte. Stride adalah lebar baris dalam byte (termasuk padding memori).
    int bytes = Math.Abs(srcData.Stride) * h;
    byte[] buffer = new byte[bytes]; // Array penampung data piksel

    // 4. Marshal.Copy: Menyalin data mentah dari memori gambar (Scan0) ke array C# (buffer)
    Marshal.Copy(srcData.Scan0, buffer, 0, bytes);

    // 5. Parallel.For: Loop multithreading.
    // Loop berjalan per 1 piksel. Karena formatnya 32bpp (ARGB), 1 piksel = 4 byte.
    // 'i' adalah indeks piksel ke-sekian.
    Parallel.For(0, bytes / 4, i =>
    {
        int idx = i * 4; // Menghitung posisi byte awal untuk piksel ini.

        // Loop 3 kali untuk channel warna (0=Blue, 1=Green, 2=Red)
        // Urutan standar Windows Bitmap adalah BGR, bukan RGB.
        for (int c = 0; c < 3; c++)
        {
            double val = buffer[idx + c]; // Ambil nilai warna asli

            // 6. RUMUS INTI: Linear Contrast Stretching
            // Menggeser nilai menjauhi titik tengah (128).
            // Jika > 128 makin terang, jika < 128 makin gelap.

```

```

        val = ((val - 128) * contrastFactor) + 128;

        // 7. Clipping/Clamping: Memastikan nilai tidak minus atau > 255.
        // Jika tidak di-clamp, warna akan "berputar" (overflow) menjadi aneh.
        buffer[idx + c] = (byte)Math.Max(0, Math.Min(255, val));
    }

    buffer[idx + 3] = 255; // Channel Alpha (Transparency). Diset 255 agar gambar tidak transparan.
});

// 8. Menyalin kembali data yang sudah dimodifikasi ke memori Bitmap tujuan
Marshal.Copy(buffer, 0, dstData.Scan0, bytes);

// 9. Melepas kunci memori (Wajib dilakukan agar tidak memory leak)
source.UnlockBits(srcData);
dst.UnlockBits(dstData);

return dst; // Kembalikan gambar hasil
}

```

Bagian 3: Method ApplyLocalContrast (Aras Lokal)

Method ini lebih canggih. Ia melakukan **Adaptive Contrast**. Untuk menentukan warna baru sebuah piksel, ia melihat **tetangga-tetangganya** (Area Window/Kernel).

C#

```

// Menerima gambar dan ukuran jendela (misal 15, berarti area 15x15 piksel)
public Bitmap ApplyLocalContrast(Bitmap source, int windowSize)
{
    if (source == null) return null;

    int w = source.Width;
    int h = source.Height;

    Bitmap dst = new Bitmap(w, h, PixelFormat.Format32bppArgb);

    // Persiapan LockBits sama seperti method sebelumnya

```

```
BitmapData srcData = source.LockBits(new Rectangle(0, 0, w, h),  
    ImageLockMode.ReadOnly, PixelFormat.Format32bppArgb);
```

```
BitmapData dstData = dst.LockBits(new Rectangle(0, 0, w, h),  
    ImageLockMode.WriteOnly, PixelFormat.Format32bppArgb);
```

```
int stride = srcData.Stride;  
int bytes = Math.Abs(stride) * h;
```

```
// Disini kita butuh DUA buffer.  
// srcBuffer untuk membaca data asli (referensi).  
// dstBuffer untuk menulis hasil. Kita tidak boleh menimpa srcBuffer karena  
// piksel tetangga butuh data asli, bukan data yang sudah diubah.  
byte[] srcBuffer = new byte[bytes];  
byte[] dstBuffer = new byte[bytes];
```

```
Marshal.Copy(srcData.Scan0, srcBuffer, 0, bytes);
```

```
// Menentukan radius. Jika window 5x5, radiusnya 2 (tengah ke pinggir).  
int radius = windowSize / 2;
```

```
// Parallel Loop berdasarkan koordinat Y (Baris)  
Parallel.For(0, h, y =>  
{  
    for (int x = 0; x < w; x++) // Loop X (Kolom)  
    {  
        // Hitung indeks array 1D berdasarkan koordinat 2D (x, y)  
        int idx = y * stride + x * 4;
```

```
        // --- TAHAP A: MENCARI MIN & MAX LOKAL ---  
        byte localMin = 255; // Inisialisasi Min dengan nilai terbesar  
        byte localMax = 0; // Inisialisasi Max dengan nilai terkecil
```

```
        // Nested Loop: Mengelilingi area tetangga (Window)  
        for (int wy = -radius; wy <= radius; wy++)  
        {  
            for (int wx = -radius; wx <= radius; wx++)  
            {  
                // Menghitung koordinat tetangga  
                int py = y + wy;  
                int px = x + wx;
```

```
                // Boundary Check: Memastikan koordinat tetangga tidak keluar dari batas gambar
```

```

        // (Misal: piksel di pojok kiri atas tidak punya tetangga kiri/atas)
        if (py >= 0 && py < h && px >= 0 && px < w)
        {
            int nIdx = py * stride + px * 4;

            // Mengambil channel Hijau (Index +1) sebagai patokan kecerahan.
            // Mengapa Hijau? Karena mata manusia paling sensitif terhadap hijau,
            // sehingga sering dipakai sebagai representasi intensitas (luminance).
            byte val = srcBuffer[nIdx + 1];

            if (val < localMin) localMin = val;
            if (val > localMax) localMax = val;
        }
    }
}

// --- TAHAP B: HITUNG FAKTOR SKALA ---
// Kita ingin merentangkan (stretch) histogram lokal agar memenuhi range 0-255.
// Rumus Scale = Range Tujuan (255) / Range Asli (Max - Min)
double scale = (localMax == localMin) ? 1 : (255.0 / (localMax - localMin));

// --- TAHAP C: TERAPKAN KE PIKSEL ---
for (int c = 0; c < 3; c++)
{
    double val = srcBuffer[idx + c];

    // Rumus Local Stretching:
    // Geser nilai agar Min menjadi 0, lalu kalikan skala agar Max menjadi 255.
    double res = (val - localMin) * scale;

    dstBuffer[idx + c] = (byte)Math.Max(0, Math.Min(255, res));
}

dstBuffer[idx + 3] = 255; // Alpha
}
});

// Salin hasil dstBuffer ke memori gambar tujuan
Marshal.Copy(dstBuffer, 0, dstData.Scan0, bytes);

source.UnlockBits(srcData);
dst.UnlockBits(dstData);

```

```
    return dst;  
}
```

Ringkasan Perbedaan Utama

Fitur	Method ApplyPointContrast	Method ApplyLocalContrast
Logic	Menggunakan rumus matematika statis (Linear) pada satu piksel.	Menganalisis statistik (Min/Max) area tetangga (<i>Windowing</i>).
Fokus	Mengubah kecerahan/kontras satu gambar secara merata.	Menonjolkan detail tekstur lokal (adaptif).
Performa	Sangat Cepat ($O(1)$ per piksel).	Lebih Lambat ($O(N*M)$ per piksel, tergantung ukuran window).
Kompleksitas	Single Pass (sekali jalan).	Nested Loop (Loop di dalam Loop).

Langkah Berikutnya

Apakah Anda ingin saya membuatkan contoh cara memanggil fungsi ini dari **Form UI (Button Click)** atau menjelaskan lebih detail tentang bagaimana Parallel.For mempercepat proses ini dibanding loop for biasa?